
Scalable Simulation-Based Inference With Optimization Monte Carlo

Anonymous Author
Anonymous Institution

Abstract

Parameter inference for complex stochastic simulators remains challenging due to intractable likelihood functions. While many simulation-based methods extend Bayesian inference to simulators, they often become particularly slow in challenging settings such as high-dimensional parameter spaces or with partially uninformative outputs, where they require large simulated datasets. We introduce a novel method for differentiable simulators that achieves accurate posterior inference in significantly faster runtimes. Our approach builds on the Optimization Monte Carlo framework by transforming the simulator into differentiable optimization problems. This enables rapid navigation to high-density posterior regions without wasting ‘blind’ simulations on low-probability areas. The method’s efficiency is further enhanced through a JAX-based implementation that parallelizes critical components. Extensive experiments show that our method effectively handles challenging scenarios, including high-dimensional parameter spaces, uninformative output dimensions, multiple observations, multimodal posteriors, and real-world applications. In these cases, it achieves comparable accuracy to existing methods while requiring substantially reduced computational time.

1 Introduction

Parametric stochastic simulators are widely used to model complex processes across biology, physics, health sciences, and many other disciplines. As simulators become more accurate, they also become more predictive. However, achieving this accuracy demands greater

complexity, such as a large numbers of free parameters or a high-dimensional output. This complexity, in turn, poses significant challenges for parameter inference.

A variety of methods, collectively known as Likelihood-Free (LFI) or Simulation-Based (SBI) Inference (Lintusaari et al., 2017; Sisson et al., 2018; Cranmer et al., 2020), have been developed to infer simulator parameters. These include sampling-based approaches, like Approximate Bayesian Computation (ABC, Marin et al., 2012), the use of summary statistics (Chen et al., 2021, 2023), parametric modeling (Wood, 2010; Price et al., 2018), and ratio estimation (Thomas et al., 2022; Durkan et al., 2020). More recently, substantial progress has come from neural-based methods, which use deep neural networks to learn summary statistics (Radev et al., 2020), or employ neural surrogate models to estimate the posterior (Greenberg et al., 2019; Papamakarios and Murray, 2016) or the likelihood function (Papamakarios et al., 2019).

Neural network-based approaches have greatly expanded the applicability of SBI to complex problem settings. However, they remain highly data-intensive; they demand massive simulated datasets to train the neural estimators and achieve satisfactory performance. As both data simulation and neural network training are expensive, neural-based methods incur significant runtimes, thereby limiting experimental flexibility and practical deployment.

This challenge is particularly pronounced in two scenarios. First, in high-dimensional parameter spaces; SBI methods suffer from the curse of dimensionality, as they rely on populating both parameter and data spaces with samples (Lintusaari et al., 2017; Sisson et al., 2018; Cranmer et al., 2020). Second, in distractor settings; In complex simulators, parameters and outputs often interact in intricate ways, so certain outputs may only depend weakly, or not at all, on certain parameters (Saltelli et al., 2008; Monsalve-Bravo et al., 2022). Such uninformative outputs act as “distractors” complicating inference (Lueckmann et al., 2021). As illustrated in Figure 1, under these scenarios existing neural-based approaches struggle when operating under a limited simulation budget.

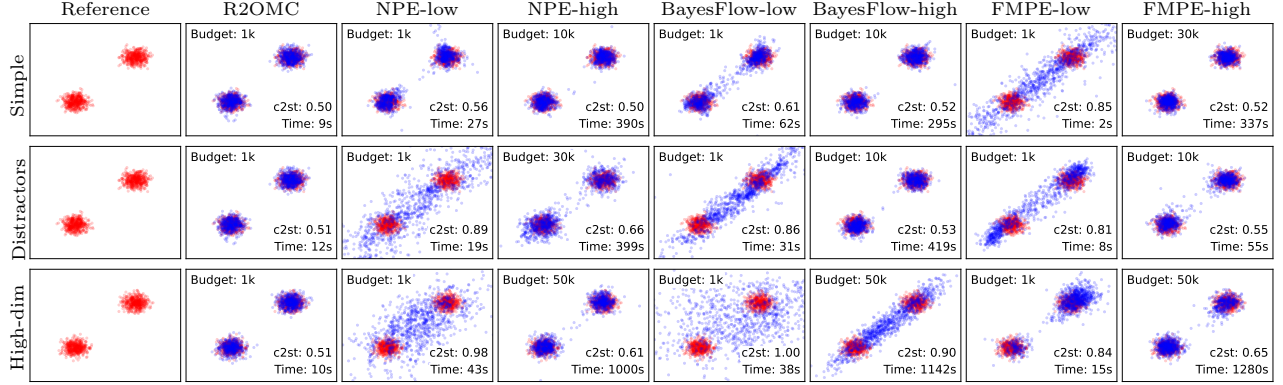


Figure 1: Top to bottom: Simple, high-dimensional, distractors, multiple observations, combined. Left to right: ground truth, NPE, NLE, Naive-ROMC, R2OMC (proposed). Details in Section C of the Appendix.

We here propose a new LFI method that uses gradient information to address these challenges. Building on the Robust Optimization Monte Carlo framework (ROMC, [Ikonov and Gutmann, 2020](#)), we reformulate the stochastic simulator as a collection of deterministic optimization problems and apply gradient-descent to obtain posterior samples. As in standard optimization, gradients are crucial for efficiently navigating in high-dimensional parameter spaces. In addition, they enable sensitivity analyses which allows us to identify and filter out distractor dimensions, thereby improving the quality of the inference.

The ROMC framework provides the theory for casting inference as deterministic optimization, but, with two crucial limitations: it lacks a strategy for defining an appropriate distance function in problems with distractors or multiple iid observations (Section 2.3), and its latest implementation ([Gkolemis et al., 2023](#)) does not exploit gradients or other computational accelerations limiting its applicability to low-dimensional problems. We address these challenges with R2OMC, an SBI method within the ROMC family, which, when the simulator is differentiable,

- enables posterior sampling in high-dimensional parameter spaces under a limited simulation budget,
- automatically identifies and filters out uninformative dimensions (distractors),
- handles-well multiple iid observations,
- runs efficiently through a JAX implementation that uses automatic differentiation, vectorization, and just-in-time compilation.

R2OMC is systematically compared to state-of-the-art SBI methods, demonstrating accurate posterior inference at a fraction of the computational cost in challenging scenarios.

2 Background and Motivation

2.1 Neural-based methods are data-intensive

Neural-based SBI have demonstrated strong performance on complex simulators. However, their effectiveness typically depends on access to extensive simulated datasets (see Appendix ?? for details on computational budgets), which makes them slow for two reasons: First, generating data from complex simulators is often costly. Second, training deep neural networks on high-volume datasets is itself time-consuming. These long runtimes are a critical limitation, as neural-based methods often require extensive hyperparameter tuning and slow runtimes limit experimental flexibility and practical deployment.

2.2 ROMC framework

ROMC has been introduced by [Ikonov and Gutmann \(2020\)](#), extending and addressing robustness issues of the Optimization Monte Carlo method by [Meeds and Welling \(2015\)](#). It belongs to the larger family of ABC methods that approximate the likelihood function as the probability of simulating an output ϵ -close to the observation,

$$L_\epsilon(\theta) = \Pr(d(\mathbf{y}, \mathbf{y}^o) \leq \epsilon \mid \theta). \quad (1)$$

Here, $d(\cdot, \cdot)$ is a distance, $\mathbf{y}$ the simulator output, and $\mathbf{y}^o$ the observed data. In the following, we denote by $g(\theta, \mathbf{u})$ the simulator that maps noise (nuisance) variables $\mathbf{u} \sim p(\mathbf{u})$ ¹ and parameters θ to multivariate data $\mathbf{y} = g(\theta, \mathbf{u})$. Introducing the acceptance region $C_\epsilon = \{(\theta, \mathbf{u}) : d(g(\theta, \mathbf{u}), \mathbf{y}^o) \leq \epsilon\}$ and using the law of the unconscious statistician, (1) can be written as

$$L_\epsilon(\theta) = \mathbb{E}_{p(\mathbf{u})} [\mathbb{1}_{C_\epsilon}(\theta, \mathbf{u})], \quad (2)$$

¹In a computer program, sampling $\mathbf{u}$ corresponds to sampling a random seed.

where $\mathbb{1}_{C_\epsilon}$ denotes the indicator function that is one if $(\boldsymbol{\theta}, \mathbf{u}) \in C_\epsilon$ and zero otherwise. When approximating the expectation as a sample average, we obtain

$$L_\epsilon(\boldsymbol{\theta}) \approx \frac{1}{S} \sum_{i=1}^S \mathbb{1}_{C_\epsilon^i}(\boldsymbol{\theta}), \quad (3)$$

which is the proportion of acceptance regions $C_\epsilon^i = \{\boldsymbol{\theta} : d(g(\boldsymbol{\theta}, \mathbf{u}_i), \mathbf{y}^o) \leq \epsilon\}$ that contain $\boldsymbol{\theta}$.

ROMC takes (3) as starting point and focuses on characterizing the acceptance regions C_ϵ^i , because once the C_ϵ^i are known, we can not only evaluate the approximate likelihood function, but also obtain posterior samples efficiently. ROMC introduces uniform proposal distributions q_i with support on C_ϵ^i only. Samples from $\boldsymbol{\theta}_{ij} \sim q_i$ for $j \in 1, \dots, M$, are then equal to posterior samples if weighted with the (unnormalized) weights w_{ij} , $i = 1, \dots, S, j = 1, \dots, M$,

$$w_{ij} = \frac{p(\boldsymbol{\theta}_{ij})}{q_i(\boldsymbol{\theta}_{ij})} \mathbb{1}_{C_\epsilon^i}(\boldsymbol{\theta}_{ij}). \quad (4)$$

For the characterization of the acceptance regions C_ϵ^i , ROMC first determines the minimizers $\boldsymbol{\theta}_i^*$,

$$\boldsymbol{\theta}_i^* = \operatorname{argmin}_{\boldsymbol{\theta}} d_i(\boldsymbol{\theta}), \quad d_i(\boldsymbol{\theta}) = d(g(\boldsymbol{\theta}, \mathbf{u}_i), \mathbf{y}^o), \quad (5)$$

where $d_i(\boldsymbol{\theta})$ are deterministic distance functions because the sources of randomness (seed) are set to $\mathbf{u}_i$ and kept fixed. Next, it starts from the optimization end points $\boldsymbol{\theta}_i^*$ to map out C_ϵ^i using for ϵ a value derived from the quantiles of the minimal distance functions, $d_i^* = d_i(\boldsymbol{\theta}_i^*)$, $i = 1, \dots, S$ (see [Ikonov and Gutmann, 2020](#), for details). While several options are possible ([Ikonov and Gutmann, 2020](#)), we here use hyperboxes centered at $\boldsymbol{\theta}_i^*$, which can be constructed with a line search algorithm for each parameter dimension. For details, please see [Appendix A](#).

The power of the ROMC framework stems from the possibility to use optimization, in particular gradient-based methods, to determine the optimization end points $\boldsymbol{\theta}_i^*$, and hence the acceptance regions C_ϵ^i and the proposal distributions q_i . We will exploit this in our method to scale SBI to higher parameter dimensions. Before we can tap into the ROMC framework, however, we will need to overcome challenges related to the definition of the distance function d in case of multiple iid observations and the presence of distractors.

2.3 ROMC challenges and limitations

ROMC suffers from three main limitations; (a) it fails in the presence of distractors, (b) it does not provide a strategy for handling multiple iid observations, and

(c) its existing implementation does not use gradients and other computational accelerations, so it can only be applied to low-dimensional problems.

Uninformative dimensions (distractors). Distractors are output dimensions that are unrelated to the parameters of interest. For example, consider a graphics renderer where the parameter of interest controls a scene element, e.g. the appearance of vase, so that most pixel values (output dimensions) are unrelated to the parameter and thus serve as distractors. To emulate distractors consider the simulator of [Figure 1](#) (second row, more details in [Appendix C](#)). There, the simulator can be written as $\mathbf{y} = (\mathbf{y}^{(1)}, \mathbf{y}^{(2)})$, where the first set of dimensions is $\mathbf{y}^{(1)} \sim 0.5 (\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I}))$, and the second $\mathbf{y}^{(2)} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I})$, where $\boldsymbol{\mu} \sim \mathcal{U}(-\mathbf{10}, \mathbf{10})$. In this setup, only $\mathbf{y}^{(1)}$ depends on $\boldsymbol{\theta}$, while $\mathbf{y}^{(2)}$ emulates distractors, so, when we observe $\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})$, the true posterior depends solely on $\mathbf{y}^{o,(1)}$. ROMC is vulnerable to failure in this setting because typical distance functions d consider all output dimensions equally. Therefore, the minimal distances d_i^* will generally be large for seeds that generate $\mathbf{y}^{(2)}$ significantly different from $\mathbf{y}^{o,(2)}$. This results in an inflated ϵ , and in turn to a much broader posterior than the ground truth.

Multiple iid observations. In many applications, we have access to multiple iid observations $\{\mathbf{y}_n^o\}_{n=1}^N$. For instance, in neuroscience, researchers often collect multiple recordings of the same neural response to a stimulus. ROMC does not prescribe a clear strategy for dealing with such cases. A straightforward approach would be to run the simulator with a different seed $\mathbf{u}_{i,n}$ for each observation n and then average the distances: $d_i(\boldsymbol{\theta}) = 1/N \sum_n \tilde{d}(g(\boldsymbol{\theta}, \mathbf{u}_{i,n}), \mathbf{y}_n^o)$, where $\tilde{d}$ is the distance for a single data point. However, this approach is sensitive to the ordering of the observations; the distance increases or decreases artificially if the observations are permuted. As before, this will generally result in artificially large minimal distances d_i^* , in an excessively large ϵ and to a much broader posterior than the ground truth. Other approaches, such as concatenating all observations into a single vector, encounter similar issues. Note that for squared Euclidean distances, the averaging and concatenation approaches are mathematically equivalent.

Computational efficiency. Current ROMC implementations ([Gkolemis et al., 2023](#)) suffer from significant computational bottlenecks that restrict their applicability to low-dimensional problems. First, optimization relies on gradient-free methods, e.g., Bayesian optimization (?), which scale poorly with the number of parameters and require many simulator evaluations.

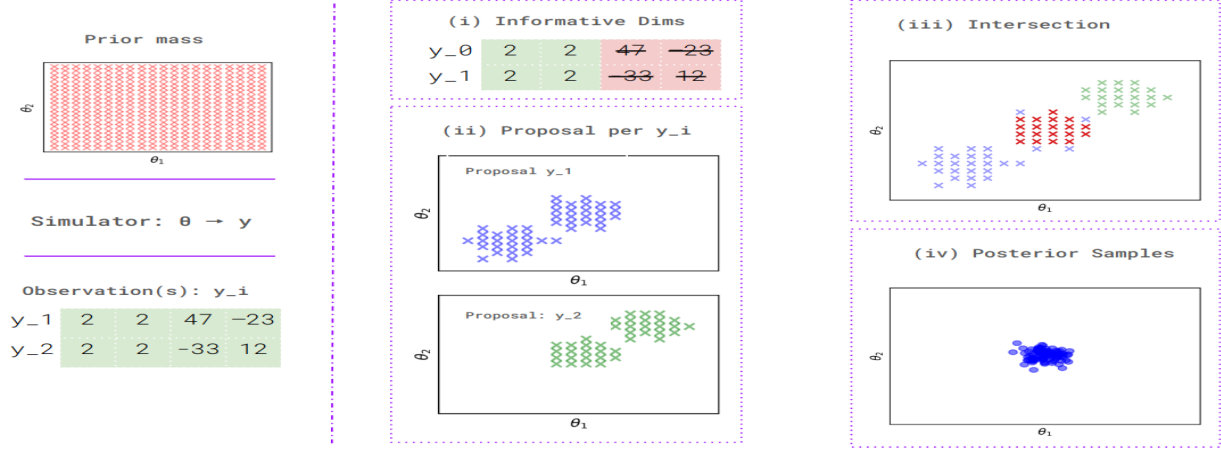


Figure 2: Overview R2OMC: Distractors are automatically filtered out and proposal distributions q_i^n are created for each observation (here two): the blue-dotted region indicates where the proposal distributions for the first observation are nonzero, and the green-dotted region indicates the same for the second observation. Only samples that fall within the intersection of both regions are given a positive weight and become posterior samples.

Second, the construction of the acceptance regions C_ϵ^i with a line-search algorithm (see Appendix A), require repeated simulator evaluations per seed and per dimension, which also scales poorly with the parameter dimension.

3 Proposed method: R2OMC

We here present our new method for Bayesian parameter inference in simulator models. The method uses gradients to obtain posterior samples even in high-dimensional parameter spaces. The method belongs to the ROMC framework so that we start with discussing how we deal with iid observations and distractors before discussing its implementation.

3.1 Uninformative dimensions (distractors)

Distractors can negatively affect the quality of the posterior approximation. To counteract their influence, we measure the sensitivity of the output dimensions with respect to the parameters θ by computing the average norm of their derivative with respect to θ , and checking that the value is above a minimal value (threshold) τ ,

$$I_i = \mathbb{E}_{p(\theta)p(\mathbf{u})} [\|\nabla_{\theta} g_i(\theta, \mathbf{u})\|] > \tau. \quad (6)$$

The expectation equals the expectation with respect to the joint distribution of the prior and the i -th dimension of the simulator output, $g_i(\theta, \mathbf{u})$. In our experiments, we approximate the expectation using 50 samples from $p(\theta)$ and $p(\mathbf{u})$ each. Collecting all binary indicator variables I_i into the vector $\mathbf{m}_\tau$, we can filter out uninformative dimensions by computing the distance between

observed and simulated data for informative dimensions only. To compute $\theta_{i,n}^*$ in (11), we minimize

$$d_i^n(\theta) = d(g(\theta, \mathbf{u}_{i,n}) \odot \mathbf{m}_\tau, \mathbf{y}_n^o \odot \mathbf{m}_\tau). \quad (7)$$

The procedure is illustrated in Figure 2, step (i).

The threshold τ controls the sensitivity of the test. In our experiments, we set τ to machine precision; more sophisticated approaches consist in monitoring the distribution of the expected norm of the gradient, or by assessing false vs true positives under control conditions. This is possible since the mask $\mathbf{m}_\tau$ can be computed prior to receiving observed data.

3.2 Multiple iid observations

The likelihood for multiple iid observations $\{\mathbf{y}_n^o\}_{n=1}^N$ is the product of the likelihoods for each data point. Approximating the likelihood for each data point as in (3) and multiplying-in the prior $p(\theta)$, we can obtain a formal expression for the approximate posterior

$$p_\epsilon(\theta | \{\mathbf{y}_n^o\}) \propto p(\theta) \prod_n \sum_i \mathbb{1}_{C_\epsilon^{i,n}}(\theta), \quad (8)$$

where $C_\epsilon^{i,n}$ is the acceptance region for data point n and source of randomness (seed) $\mathbf{u}_{i,n}$,

$$C_\epsilon^{i,n} = \{\theta : d(g(\theta, \mathbf{u}_{i,n}), \mathbf{y}_n^o) \leq \epsilon\}. \quad (9)$$

We have S sources of randomness for each data point $\mathbf{y}_n^o$, and so in total $S * N$ acceptance regions $C_\epsilon^{i,n}$.

We now address the question how to efficiently sample from $p_\epsilon(\theta | \{\mathbf{y}_n^o\})$. To achieve this, we consider the task of computing the posterior expectation $\bar{h} = \mathbb{E}_{p_\epsilon}[h(\theta)]$

under $p_\epsilon(\boldsymbol{\theta}|\{\mathbf{y}_n^o\})$. Plugging in the expression in (8) gives, after normalization,

$$\bar{h} = \frac{\int h(\boldsymbol{\theta}) p(\boldsymbol{\theta}) \prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}) d\boldsymbol{\theta}}{\int p(\boldsymbol{\theta}) \prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}) d\boldsymbol{\theta}}. \quad (10)$$

The integrals in the equation are typically intractable. But they correspond to expectations with respect to the prior $p(\boldsymbol{\theta})$ and hence could be approximated with a sample average. However, this is very inefficient. Few (or no) prior samples will generate outputs ϵ -close to all observations, so most (or all) will get zero weight, leading to a very low effective sample size. Following the general ROMC framework by [Ikonov and Gutmann \(2020\)](#), we thus employ importance sampling with an adaptive proposal distribution obtained via optimization.

We first treat each data point $\mathbf{y}_n^o$ separately and construct proposal distributions $q_i^n(\boldsymbol{\theta})$, $i = 1, \dots, S$, as in Section 2.2: We use optimization to find the minimizers $\boldsymbol{\theta}_{i,n}^*$ of the deterministic functions $d_i^n(\boldsymbol{\theta}) = d(g(\boldsymbol{\theta}, \mathbf{u}_{i,n}), \mathbf{y}_n^o)$,

$$\boldsymbol{\theta}_{i,n}^* = \operatorname{argmin}_{\boldsymbol{\theta}} d_i^n(\boldsymbol{\theta}), \quad (11)$$

and then build a hyperbox around each acceptance region $C_\epsilon^{i,n}$ and define $q_i^n(\boldsymbol{\theta})$ to be the uniform distribution over it. The optimization problem in (11) is deterministic and we solve it with gradient-based methods to enable scalability to large parameter spaces. Note that treating each data point $\mathbf{y}_n^o$ separately allows us to avoid the issues explained in Section 2.3.

We propose to combine the per-data point proposal distributions $q_i^n(\boldsymbol{\theta})$ in form of a mixture distribution,

$$q(\boldsymbol{\theta}) = \frac{1}{N} \frac{1}{S} \sum_{n=1}^N \sum_{i=1}^S q_i^n(\boldsymbol{\theta}). \quad (12)$$

The posterior expectation in (10) can then be approximated in form of a weighted average

$$\bar{h} = \frac{\sum_p w_p h(\boldsymbol{\theta}_p)}{\sum_p w_p}, \quad \boldsymbol{\theta}_p \sim q(\boldsymbol{\theta}) \quad (13)$$

where the weights w_p , $p = 1, \dots, P$, are given by

$$w_p = \frac{p(\boldsymbol{\theta}_p)}{q(\boldsymbol{\theta}_p)} \prod_{n=1}^N \sum_{i=1}^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}_p). \quad (14)$$

This means that the $\boldsymbol{\theta}_p \sim q(\boldsymbol{\theta})$, weighted by w_p , represent samples from the posterior $p_\epsilon(\boldsymbol{\theta}|\{\mathbf{y}_n^o\})$ in (8).

The rationale for choosing $q(\boldsymbol{\theta})$ in (12) as proposal distribution is as follows: it is a mixture of uniform distributions, so sampling and evaluating is easy. It

seamlessly combines the contribution from multiple data points, allowing future data points to be included without re-computation. If each q_i^n encloses the corresponding acceptance region $C_\epsilon^{i,n}$, then $q(\boldsymbol{\theta}) > 0$ in areas of the parameter space where $\prod_n^N \sum_i^S \mathbb{1}_{C_\epsilon^{i,n}}(\boldsymbol{\theta}) > 0$ and no posterior mass is discarded in the weighting in (14). Finally, regions of high likelihood are characterized by regions where multiple $C_\epsilon^{i,n}$ overlap. As the proposal distribution $q(\boldsymbol{\theta})$ consists of a mixture of the $q_i^n(\boldsymbol{\theta})$ with equal weights, areas where multiple $C_\epsilon^{i,n}$ overlap will be sampled more often, so that areas of high likelihood will be represented with more samples.

Figure 2, steps (ii-iii), illustrates the proposed procedure to obtain posterior samples. We have two observed data points and the $\boldsymbol{\theta}_p$ come from regions that match at least one observation; the blue-dotted area for $\mathbf{y}_1^o$ and the green-dotted region for $\mathbf{y}_2^o$. However, only the ones that match both will get a positive weight (red dots) and become posterior samples.

3.3 JAX implementation

Algorithms 1 summarizes the key steps of R2OMC. It has three steps: dealing with distractors (line 2); constructing per-data point proposal distributions $\{q_i^n\}_{i=1}^S$ (lines 4-5); combining them to form the overall proposal distribution $q(\boldsymbol{\theta})$ and generating the weighted samples (lines 7-8).

Algorithm 1 R2OMC. Requires: $p(\boldsymbol{\theta}), g(\boldsymbol{\theta}, \mathbf{u}), \{\mathbf{y}_n^o\}_{n=1}^N$

```

1: procedure R2OMC( $\tau, S, P$ )
2:    $\mathbf{m}_\tau = (I_1, \dots, I_{D_y})$  using (6) with  $\tau$ 
3:   for  $n \leftarrow 1$  to  $N$  do
4:      $\boldsymbol{\theta}_{i,n}^* = \operatorname{argmin}_{\boldsymbol{\theta}} d_i^n(\boldsymbol{\theta})$  using (7) for all  $i$ 
5:     Compute hyperboxes and  $\{q_i^n\}_{i=1}^S$ 
6:   end for
7:    $\boldsymbol{\theta}_p \sim q(\boldsymbol{\theta})$ , where  $q(\boldsymbol{\theta}) = \frac{1}{N} \frac{1}{S} \sum_n \sum_i q_i^n(\boldsymbol{\theta})$ 
8:   Compute  $\{w_p\}_p^P$  using (14)
9:   return  $\{w_p, \boldsymbol{\theta}_p\}_{p=1}^P$ 
10: end procedure
```

We offer an efficient JAX implementation, benefiting from automatic differentiation (`grad`), vectorization (`vmap`), and just-in-time compilation (`jit`). In many cases, R2OMC executes in seconds where most SBI methods require minutes (see Section 4).

R2OMC requires approximately $N \cdot S$ evaluations of the simulator: For the distractors, `grad` computes $\partial g_i / \partial \theta_j$ for all i and j in one call, and `vmap` evaluates multiple $\boldsymbol{\theta}$ at once. Combining them, the cost is S . Obtaining the per-data point proposal distributions $\{q_i^n\}_{i=1}^S$ consists of optimization (line 4) and hyperbox building (line 5) for each data point. Each of S optimizations requires T gradient descent steps, where T is a user-defined

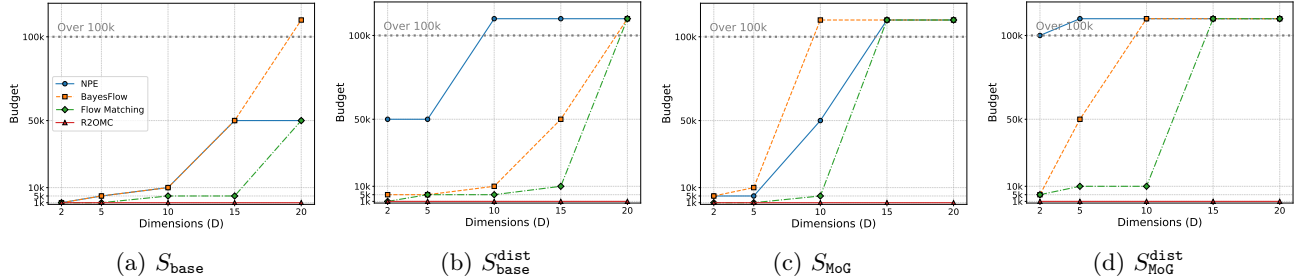


Figure 3: MoG benchmark: Success Frontier plots showing the lowest budget required to reach a C2ST score ≤ 0.75 for varying D .

hyperparameter. `jit` allows compiling T_1 consecutive steps to apply them at the cost of a single execution, so the cost reduces to $S \frac{T}{T_1}$. $\frac{T}{T_1}$ is normally small, and can even reach 1 if memory allows setting $T_1 = T$. Hyperbox building also requires of the order of S steps when the required line searches are parallelized over each parameter dimensions using `vmap` (see Appendix A). Thus the total cost of this step is of the order of $N * S$. For generating the weighted samples, (14) is evaluated on P samples (line 8). The simulator is only used for computing $\prod_n \sum_i^S \mathbb{1}_{C_e^{i,n}}(\theta_p)$ which requires evaluating $S * N$ different distance functions on P points each. Using `vmap` in each distance function, the total cost is $S * N$. Evaluating $q(\cdot)$ and $p(\cdot)$ does not require calling the simulator. Finally, if the simulator is expensive, the $S * N$ cost of checking whether the samples are in the acceptance regions can be avoided by using the hyperbox or another model for the acceptance region, see [Ikononov and Gutmann \(2020\)](#).

The code is available at [anonymized link](#).

4 Experiments

To assess R2OMC, we conduct experiments on a Mixture of Gaussians benchmark, on the synthetic SBI benchmark and an image-based inference problem, covering scenarios with high-dimensional data, distractors, multiple observations and complex posterior shapes.

Methods. In the MoG benchmark, we compare R2OMC against three well-known neural methods: NPE ([Greenberg et al., 2019](#)), BayesFlow ([Radev et al., 2020](#)) and FlowMatching ([Wildberger et al., 2023](#)). We use our JAX implementation for R2OMC and for the competitors the implementation provided by the SBI Pytorch package. In the SBI benchmark, we compare R2OMC against all methods reported by [Lueckmann et al. \(2021\)](#). In the image-based inference problem, we illustrate the applicability of R2OMC to a high-dimensional real-world problem, without comparing it. All experiments were conducted on a single Dell XPS

PC equipped with an Intel® Core™ i7-10750H CPU @ 2.60GHz with 12 cores. Appendix D contains further details on the setup and methods.

Metrics. All methods are evaluated with the C2ST score ([Gutmann et al., 2018; Lueckmann et al., 2021](#)), which uses classification to measure the similarity between two sets; the first set are the samples from the approximate posterior and the second set from the ground truth. The best score is 0.5, indicating that the two sets are indistinguishable, while the worst score is 1.0, indicating perfect separation.

4.1 MoG benchmark

We systematically evaluate SBI methods when the dimensionality of the parameter space D increases. We use two simulators, $S_{\text{base}} : \mathbf{y} \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{I})$ and $S_{\text{MoG}} : \mathbf{y} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2 \mathbf{I}))$, that lead to uni-modal and bi-modal posteriors respectively. For each simulator, we consider a simple version, where the output has the same dimensionality as the parameter vector ($D_y = D$), and a distractor version, where we append 100 uninformative dimensions to the output ($D_y = D + 100$). The distractor dimensions are sampled from $\mathcal{N}(\boldsymbol{\mu}, 100 \mathbf{I})$, with $\boldsymbol{\mu} \sim \mathcal{U}(-10, 10)$. That is, we consider four simulators in total.

As observed data, we use $\mathbf{y}_n^o \approx (5, \dots, 5)$, that is a vector of 5’s with some small noise around it. In case of distractors, we append 100 zeros to it. The posterior for S_{base} is then approximated by a truncated Gaussian, while the posterior for S_{MoG} is well approximated by a truncated bi-modal Gaussian.

For each simulator, we evaluate R2OMC against the three neural methods mentioned above, varying D from 2 to 20 and the simulation budget from 1_000 to 100_000. For all simulators, we use $\mathcal{U}(-3, 3)$ as prior. For more details on the setup, see Appendix D.

Figure 3 summarizes the results in form of Success Frontier plots, showing the lowest budget required for a successful inference, defined as reaching a C2ST score

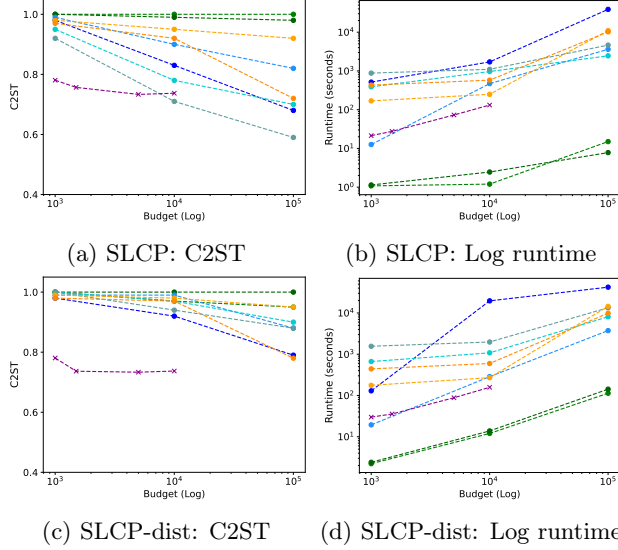


Figure 4: SLCP and SLCP with distractors benchmark results.

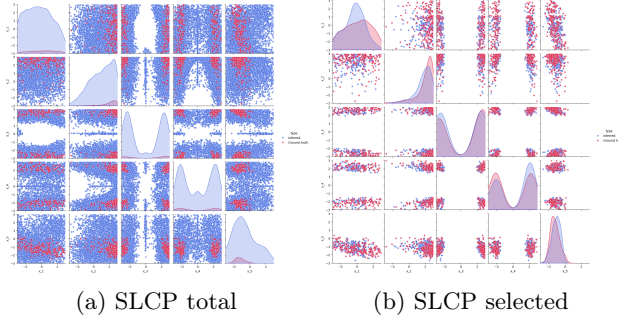


Figure 5: SLCP pairwise posteriors

≤ 0.75 , for varying D . All neural-based methods follow the same trend; as D increases, so does the required budget, and soon they ask for more than 100_000 simulations to succeed, which incurs very high runtimes (see Appendix D). They struggle more with S_{MoG} than with S_{base} , as the posterior is more complex and more on the distractor versions than on the simple ones. In contrast, R2OMC succeeds with a budget of up to 1_000 simulations and a runtime of a few seconds for all D and all simulators.

4.2 SBI benchmark

The SBI (Lueckmann et al., 2021) is a widely used benchmark for SBI methods. We select two characteristic experiments from the benchmark that cover different challenges.

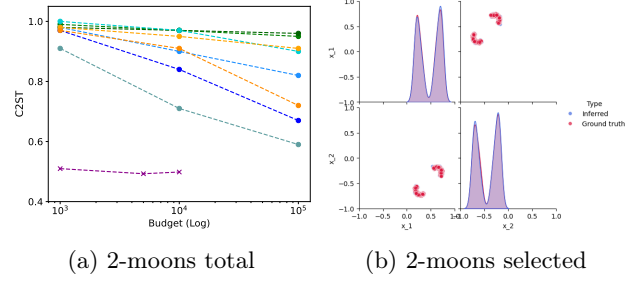


Figure 6: 2-moons pairwise posteriors

4.2.1 SLCP (T.3) and plus distractors (T.4)

As its name suggests, the SLCP task is designed to have a Simple Likelihood but a Complex Posterior (Papamakarios et al., 2019; Greenberg et al., 2019).

The prior is uniform over five parameters $\theta \in \mathcal{U}(-3, 3)^5$ and the observations consist of four two-dimensional points drawn from a Gaussian distribution whose mean and variance are nonlinear functions of θ . This setup yields a posterior with four symmetric modes and sharp vertical cut-offs. SLCP with distractors augments the original SLCP task by adding 92 additional non-informative outputs, resulting in a total of 100 output dimensions.

Figure 4 illustrates the C2ST score and runtime of R2OMC and all methods reported by Lueckmann et al. (2021) for SLCP (top row) and SLCP with distractors (bottom row). R2OMC achieves an accurate posterior approximation (C2ST score between 0.7 and 0.8) with a small simulation budget of 10^3 samples and a runtime of a few seconds. All other methods require a budget of at least 10^4 (and often 10^5) samples to reach a similar score, leading to runtimes of several minutes or even hours. The results are more pronounced in the distractor version, where most methods fail to reach a C2ST score below 0.8 even with a budget of 10^5 samples.

Figure 5 illustrates the rationale behind the multiple observation weighting scheme in R2OMC (Eq. 14). Figure 5a shows the pairwise posteriors using all proposal samples generated by R2OMC, while Figure 5b using only the ones that are finally selected. The proposal samples are the ones that are in ϵ -distance to at least one observation, while the selected samples are the ones that are in ϵ -distance to all observations and thus represent the posterior.

4.2.2 Two-moons (T.8)

The 2-moons task tests whether the methods can accurately estimate a posterior with both global (bimodality) and local (crescent shape) structure. The simulator is $\mathbf{y}|\theta \sim (r \cos(\alpha) + 0.25, r \sin(\alpha)) + (-|\theta_1| +$

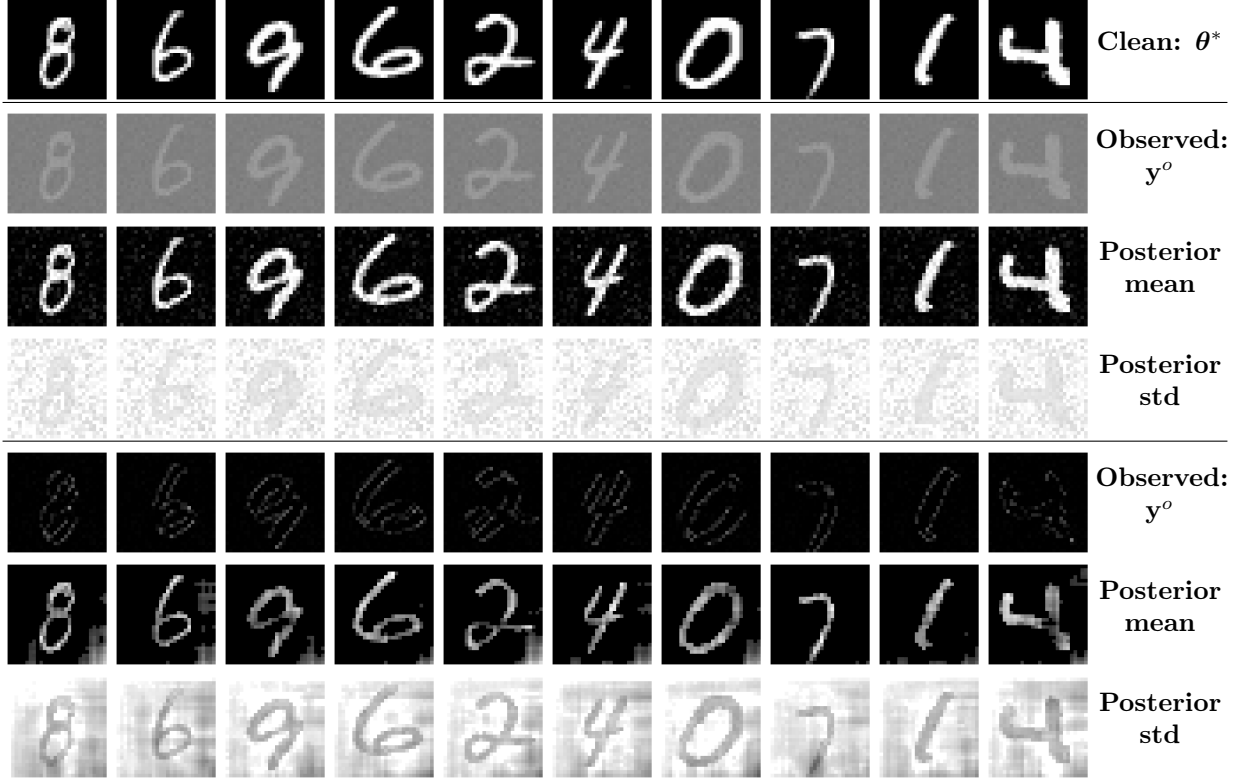


Figure 7: Your figure caption here.

$\theta_2|/\sqrt{2}, (-\theta_1 + \theta_2)/\sqrt{2})$, $\alpha \sim \mathcal{U}(-\pi/2, \pi/2)$, $r \sim \mathcal{N}(0.1, 0.01^2)$. The parameters $\theta = (\theta_1, \theta_2)$ are two-dimensional and the prior is $\theta \sim \mathcal{U}(-1, 1)$. This creates a posterior with two half-moon shape modes.

Figure 6 shows the C2ST score (left) and the pairwise posterior (right) of R2OMC and all methods reported by Lueckmann et al. (2021). R2OMC has a C2ST score close to 0.5 even with a simulation budget of 10^3 samples, while the rest of the methods need a budget of 10^5 or more to achieve a similar score and perform much worse for smaller budgets. The pairwise posterior in Figure 6b shows that R2OMC accurately captures the two crescent-shaped modes of the posterior.

4.3 Image-based inference

We consider a high-dimensional image-based inference problem, where the goal is to infer the parameters of a random simulator that distorts and adds Gaussian noise to an input image. The problem tests the ability of R2OMC to handle very high-dimensional data ($D_y = 28 \times 28 = 784$), even beyond the dimensionality of conventional SBI problems. The simulator is $y = \text{transform}(\theta) + \epsilon$, where $\epsilon \sim \mathcal{N}(\mathbf{0}, 0.1^2 \mathbf{I})$ and $\text{transform}(\theta)$ applies a transformations to the base image. We test two types of transformations: (a) pixel-wise intensity changes, where $\theta \in \mathbb{R}^{28 \times 28}$ and each

parameter controls the intensity of one pixel; (b) a checkerboard filter, where an edge-detection filter is applied to the base image.

Figure 7 summarizes the results. In the top row, we show the initial clean image. In the second row the pixel-wise distorted observation and in the third and fourth rows the posterior mean and standard deviation obtained by R2OMC(case (a)). In the last three rows, we show the observation, posterior mean and standard deviation for the checkerboard filter (case (b)). In both cases, R2OMC obtains a posterior mean that is relatively close to the clean image, in terms of optical similarity, which means that the posterior mode is close to parameter θ^* that generated the observation.

5 Conclusion

We have presented R2OMC, a new Bayesian inference method for simulator models that addresses three key scalability challenges in simulation-based (likelihood-free) inference, namely high-dimensional parameter spaces, multiple observations, and the possible presence of uninformative dimensions (i.e., distractors). R2OMC builds on the ROMC framework and formulates the inference problem in terms of a set of deterministic optimization problems that can be solved with gradient-descent, enabling inference in high-dimensional param-

ter spaces. It further introduces a gradient-based filtering step to mask distractors during inference. It uses a novel adaptive importance sampling approach that efficiently identifies posterior samples that explain *all* available iid observations. We offer an efficient **JAX**-based implementation that leverages key auto-differentiation and vectorization features to accelerate the inference. Simulations performed with novel experimental settings as well as the comprehensive SBI benchmark show that R2OMC achieves improved accuracy (close to the optimal 0.5 C2ST) compared to existing methods, while using a smaller number of simulator calls and a fraction of the computational time.

References

- Jarno Lintusaari, Michael U Gutmann, Ritabrata Dutta, Samuel Kaski, and Jukka Corander. Fundamentals and recent developments in approximate bayesian computation. *Systematic biology*, 66(1): e66–e82, 2017.
- Scott A Sisson, Yanan Fan, and Mark Beaumont. *Handbook of approximate Bayesian computation*. CRC press, 2018.
- Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48): 30055–30062, 2020.
- Jean-Michel Marin, Pierre Pudlo, Christian P. Robert, and Robin J. Ryder. Approximate bayesian computational methods. *Statistics and Computing*, 22(6):1167–1180, 2012. ISSN 1573-1375. doi:[10.1007/s11222-011-9288-2](https://doi.org/10.1007/s11222-011-9288-2). URL <https://doi.org/10.1007/s11222-011-9288-2>.
- Y. Chen, D. Zhang, M. U. Gutmann, A. Courville, and Z. Zhu. Neural approximate sufficient statistics for implicit models. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=SRDuJssQud>.
- Yanzhi Chen, Michael U. Gutmann, and Adrian Weller. Is learning summary statistics necessary for likelihood-free inference? In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 4529–4544. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/chen23h.html>.
- Simon N Wood. Statistical inference for noisy nonlinear ecological dynamic systems. *Nature*, 466(7310):1102–1104, 2010.
- Leah F Price, Christopher C Drovandi, Anthony Lee, and David J Nott. Bayesian synthetic likelihood. *Journal of Computational and Graphical Statistics*, 27(1):1–11, 2018.
- Owen Thomas, Ritabrata Dutta, Jukka Corander, Samuel Kaski, and Michael U Gutmann. Likelihood-free inference by ratio estimation. *Bayesian Analysis*, 17(1):1–31, 2022.
- Conor Durkan, Iain Murray, and George Papamakarios. On contrastive learning for likelihood-free inference. In *International conference on machine learning*, pages 2771–2781. PMLR, 2020.
- Stefan T Radev, Ulf K Mertens, Andreas Voss, Lynton Ardizzone, and Ullrich Köthe. Bayesflow: Learning complex stochastic models with invertible neural networks. *IEEE transactions on neural networks and learning systems*, 33(4):1452–1466, 2020.
- David Greenberg, Marcel Nonnenmacher, and Jakob Macke. Automatic posterior transformation for likelihood-free inference. In *International Conference on Machine Learning*, pages 2404–2414. PMLR, 2019.
- George Papamakarios and Iain Murray. Fast ϵ -free inference of simulation models with bayesian conditional density estimation. *Advances in neural information processing systems*, 29, 2016.
- George Papamakarios, David Sterratt, and Iain Murray. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd international conference on artificial intelligence and statistics*, pages 837–848. PMLR, 2019.
- A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, and S. Tarantola. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, 2008.
- Gloria M. Monsalve-Bravo, Brodie A. J. Lawson, Christopher Drovandi, Kevin Burrage, Kevin S. Brown, Christopher M. Baker, Sarah A. Vollert, Kerrie Mengersen, Eve McDonald-Madden, and Matthew P. Adams. Analysis of sloppiness in model simulations: Unveiling parameter uncertainty when mathematical models are fitted to data. 8(38): eabm5952, 2022. doi:[10.1126/sciadv.abm5952](https://doi.org/10.1126/sciadv.abm5952). URL <https://www.science.org/doi/abs/10.1126/sciadv.abm5952>.
- Jan-Matthis Lueckmann, Jan Boelts, David Greenberg, Pedro Goncalves, and Jakob Macke. Benchmarking simulation-based inference. In *International conference on artificial intelligence and statistics*, pages 343–351. PMLR, 2021.
- Borislav Ikononov and Michael U Gutmann. Robust optimisation monte carlo. In *International Conference on Artificial Intelligence and Statistics*, pages 2819–2829. PMLR, 2020.
- Vasilis Gkolemis, Michael Gutmann, and Henri Pesonen. An extendable python implementation of robust optimisation monte carlo. *arXiv preprint arXiv:2309.10612*, 2023.
- Ted Meeds and Max Welling. Optimization monte carlo: Efficient and embarrassingly parallel likelihood-free inference. *Advances in Neural Information Processing Systems*, 28, 2015.
- Jonas Wildberger, Maximilian Dax, Simon Buchholz, Stephen Green, Jakob H Macke, and Bernhard Schölkopf. Flow matching for scalable simulation-based inference. *Advances in Neural Information Processing Systems*, 36:16837–16864, 2023.

Alvaro Tejero-Cantero, Jan Boelts, Michael Deistler, Jan-Matthis Lueckmann, Conor Durkan, Pedro J. Gonçalves, David S. Greenberg, and Jakob H. Macke. sbi: A toolkit for simulation-based inference. *Journal of Open Source Software*, 5(52):2505, 2020. doi:[10.21105/joss.02505](https://doi.org/10.21105/joss.02505). URL <https://doi.org/10.21105/joss.02505>.

Michael U Gutmann, Ritabrata Dutta, Samuel Kaski, and Jukka Corander. Likelihood-free inference via classification. *Statistics and Computing*, 28:411–425, 2018.

Jarno Lintusaari, Henri Vuollekoski, Antti Kangasraasio, Kusti Skytén, Marko Jarvenpaa, Pekka Marttinen, Michael U Gutmann, Aki Vehtari, Jukka Corander, and Samuel Kaski. Elfi: Engine for likelihood-free inference. *Journal of Machine Learning Research*, 19(16):1–7, 2018.

Checklist

The checklist follows the references. For each question, choose your answer from the three possible options: Yes, No, Not Applicable. You are encouraged to include a justification to your answer, either by referencing the appropriate section of your paper or providing a brief inline description (1-2 sentences). Please do not modify the questions. Note that the Checklist section does not count towards the page limit. Not including the checklist in the first submission won't result in desk rejection, although in such case we will ask you to upload it during the author response period and include it in camera ready (if accepted).

In your paper, please delete this instructions block and only keep the Checklist section heading above along with the questions/answers below.

1. For all models and algorithms presented, check if you include:
 - (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes]
 - (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes]
 - (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [No]
2. For any theoretical claim, check if you include:
 - (a) Statements of the full set of assumptions of all theoretical results. [Not Applicable]

- (b) Complete proofs of all theoretical results. [Not Applicable]
 - (c) Clear explanations of any assumptions. [Not Applicable]
3. For all figures and tables that present empirical results, check if you include:
 - (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [No] Not now, but code will be made available if accepted.
 - (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes]
 - (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes]
 - (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [Yes]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
 - (a) Citations of the creator If your work uses existing assets. [Yes]
 - (b) The license information of the assets, if applicable. [Yes]
 - (c) New assets either in the supplemental material or as a URL, if applicable. [Not Applicable]
 - (d) Information about consent from data providers/curators. [Not Applicable]
 - (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Not Applicable]
5. If you used crowdsourcing or conducted research with human subjects, check if you include:
 - (a) The full text of instructions given to participants and screenshots. [Not Applicable]
 - (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable]
 - (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable]

A Methods

Throughout the paper we run experiments based on four methods: R2OMC (our proposal), NAIVE-ROMC (a straightforward use of the ROMC framework by [Ikonov and Gutmann, 2020](#)), Neural Posterior Estimation (NPE) ([Greenberg et al., 2019](#)), Neural Likelihood Estimation (NLE) ([Papamakarios et al., 2019](#)).

R2OMC. A detailed description of the method can be found in Algorithm 1.

In Step 2 of Algorithm 1, i.e., computing the uninformative dimensions, we typically use around 50 samples from the distribution $p(u)$ and another 50 samples from $p(\theta)$, with the parameter τ set to machine precision, i.e., $\tau \approx 0$. Similar results can be obtained with fewer samples, such as 10 or 20.

For Step 3, i.e., computing the optimal points $\theta_{i,n}^*$, we use the Adam optimizer, with a learning rate between 0.01 and 0.2, depending on the specific problem. Our default choice is 0.1. The number of optimization steps varies between 100 and 400, with 50 as the default. The distance function $d(\theta)$ is defined as the squared Euclidean distance between the simulator output and the observation, i.e., $d(\theta) = \|\mathbf{y} - \mathbf{y}_n^o\|_2^2$. The number of seeds S used ranges from 500 to 5000, with 1000 as the default. The number of seeds corresponds to the simulation budget; for example, if a budget of 10,000 runs is available, we use (at most) 10,000 seeds. Typically, we select 80% of the seeds based on the best $d_i^n(\theta_{i,n}^*)$. However, the percentage of accepted seeds can vary between 50% and 100%, depending on the absolute values of $d_i^n(\theta_{i,n}^*)$ for $i = 1, \dots, S$.

For Step 5, i.e., constructing the hyperboxes, we use Algorithm 2. We normally set ϵ to be twice the distance of the ‘worst’ accepted seed, i.e., $\epsilon = 2 \max_i d_i^n(\theta_{i,n}^*)$, where i is an index over the accepted seeds. Alternatively, ϵ can be set to a fixed value such as 0.1. The typical settings for step size, number of steps, and refinements are $\eta = 0.1$, $L = 100$, and $R = 1$, respectively.

For Step 7, i.e., sampling from the proposal distribution, we generate between 1000 and 100,000 candidate samples, with the default being 5000. From these candidates, we select 1000 via weighted sampling. For computing the weights, $\{w_p\}_{p=1}^P$, we set ϵ to a value about twice the distance of the ‘worst’ accepted seed, i.e., $\epsilon = 2 \max_{i,n} d_i^n(\theta_{i,n}^*)$. However, this can vary depending on the problem, and by testing different values.

Finally, we keep 1000 posterior samples, which are used to compute the C2ST score using another 1000 samples drawn from the true posterior.

Algorithm 2 Hyperbox Computation

Input: Optimal point θ^* , distance function $d(\theta)$, Jacobian $\mathbf{J}$ at θ^*

Parameters: Step size η , number of refinements R , number of steps L , distance threshold ϵ

- 1: Compute eigenvectors $\mathbf{v}_d$ of $\mathbf{J}^T \mathbf{J}$ ($d = 1, \dots, D$)
 - 2: **for** $d = 1$ to D **do**
 - 3: Initialize endpoint $\tilde{\theta} \leftarrow \theta^*$
 - 4: Set refinement counter $r \leftarrow 0$
 - 5: **repeat**
 - 6: Set step counter $l \leftarrow 0$
 - 7: **repeat**
 - 8: Increment step counter $l \leftarrow l + 1$
 - 9: Update $\tilde{\theta} \leftarrow \tilde{\theta} + \eta \mathbf{v}_d$
 - 10: **until** $d(\tilde{\theta}) > \epsilon$ or $l = L$
 - 11: Move back one step $\tilde{\theta} \leftarrow \tilde{\theta} - \eta \mathbf{v}_d$
 - 12: Halve the step size $\eta \leftarrow \eta/2$
 - 13: Increment refinement counter $r \leftarrow r + 1$
 - 14: **until** $r = R$
 - 15: Set $\tilde{\theta}$ as the endpoint
 - 16: Repeat steps 3-15 for $\mathbf{v}_d = -\mathbf{v}_d$ and set $\tilde{\theta}$ as the negative endpoint
 - 17: **end for**
 - 18: Define a uniform distribution q over the hyperbox
 - 19: **return** q
-

NAIVE-ROMC. We use the Algorithm presented as ROMC in [Ikonov and Gutmann \(2020\)](#). The characterization NAIVE indicates the use of a naive distance function in case of multiple observations. As described in the main paper, in case of N observations, we concatenate them in a single vector $\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \dots, \mathbf{y}^{o,(N)})$, then, run the simulator N times, concatenate the outputs, $\mathbf{y} = (\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)})$ and set the distance to be the squared Euclidean distance between the two vectors, $d(\boldsymbol{\theta}) = \|\mathbf{y} - \mathbf{y}^o\|_2^2$. Although using this approach for defining $d(\boldsymbol{\theta})$ has disadvantages, that is why we name it ‘NAIVE’, there is no other (simple) approach that leads to a better posterior approximation, under multiple observations.

NAIVE-ROMC is practically the same as R2OMC (Algorithm 1 of the main text), if (a) we omit Step 2, i.e., computing the uninformative dimensions and (b) set $N = 1$. This is why NAIVE-ROMC and R2OMC have a similar C2ST score in problems without distractors and with a single observation. In case of multiple observations, we use as $\mathbf{y}$ and $\mathbf{y}^o$, the concatenated vectors as described above. For NAIVE-ROMC, we use a JAX-based implementation that is similar to the one we use for R2OMC. The reason for not using its existing implementation ([Gkolemis et al., 2023](#)) of the ELFI package ([Lintusaari et al., 2018](#)) is that it does not use gradients, so it cannot scale to high dimensions. The hyperparameters used in NAIVE-ROMC are similar to R2OMC, as described above.

NPE As Neural Posterior Estimation (NPE) we use the one denoted as SNPE_C in the SBI benchmark ([Lueckmann et al., 2021](#)), which is a method proposed by [Greenberg et al. \(2019\)](#). SNPE_C estimates the posterior by training a neural network $F(\mathbf{y}, \phi)$ to approximate $p(\boldsymbol{\theta}|\mathbf{y})$ using a density estimator $q_F(\mathbf{y}, \phi)(\boldsymbol{\theta})$. For further details, see ([Greenberg et al., 2019](#)).

We use the Pytorch implementation provided by the SBI package, using mostly the default parameters. As density estimator we use the a neural network with 50 hidden units, `sbi.utils.get_nn_models.posterior_nn()` with parameters `model='nsf'`, `hidden_features=50`, `z_score_x='independent'`, and `z_score_y='independent'`. During training, we use 10 atoms, a batch size of 1000 (or the whole dataset) and a maximum number of 500 epochs.

We use a simulation budget of 10,000 simulator executions, which means that we train the density estimator on 10,000 samples. We normally use 2 training rounds, each with 5000 training samples: the first set of 5000 is drawn from the prior, while the second set is sampled from the approximate posterior learned after the first round. Occasionally, if the initial 5000 samples do not yield a reliable approximation and conditioning on $\mathbf{y}^o$ leads to instabilities, we use a single round of 10000 samples.

In cases with multiple observations, since NPE cannot handle them separately, we follow the same approach as in NAIVE-ROMC(see above); we concatenate the observations and the outputs to one vector each.

NLE As NLE we use the method proposed by [Papamakarios et al. \(2019\)](#). NLE approximates the likelihood $p(\mathbf{y}^o|\boldsymbol{\theta})$ using neural density estimation. The process involves generating a dataset $D = \{(\boldsymbol{\theta}_k, \mathbf{y}_k)\}_{k=1}^K$ by sampling $\boldsymbol{\theta}_k \sim p(\boldsymbol{\theta})$ and simulating $\mathbf{y}_k \sim p(\mathbf{y}|\boldsymbol{\theta}_k)$. A conditional neural density estimator $q_\psi(\mathbf{y}|\boldsymbol{\theta})$ is then trained to model the likelihood.

Once trained, samples from the posterior $\hat{p}(\boldsymbol{\theta}|\mathbf{y}^o) \propto q_\psi(\mathbf{y}^o|\boldsymbol{\theta})p(\boldsymbol{\theta})$ are obtained using MCMC. For density estimation, we use the default SBI settings, i.e., a Masked Autoregressive Flow (MAF) with five flow transforms and 50 hidden units per block. For MCMC, 250 warm-up steps and posterior samples are obtained with thinning.

We use a simulation budget of 10,000 simulator executions, which means that we train the density estimator on 10,000 samples. We always use a single training round. NLE can handle multiple observations; we simply pass a numpy array of shape: (N,D) , where N is the number of observations and D their dimensionality. Internally, NLE evaluates each observation using the neural density estimator $q_\psi(\mathbf{y}|\boldsymbol{\theta})$ and multiplies them to estimate the joint likelihood and then uses MCMC to sample from the posterior.

B Discussion about simulation budget

??

TODO

C Introductory Example

Notation. We use $\mathcal{N}(\mathbf{y}; \boldsymbol{\theta}, \Sigma)$ to denote the probability density function (pdf) of a Gaussian random variable $\mathbf{y}$, while $\mathcal{N}(\boldsymbol{\theta}, \Sigma)$ denotes the random variable itself. Typically, $\mathcal{N}(\mathbf{y}; \boldsymbol{\theta}, \Sigma)$ is used when deriving expressions, whereas $\mathcal{N}(\boldsymbol{\theta}, \Sigma)$ is used to indicate sampling, such as in $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\theta}, \Sigma)$. We use $\mathcal{U}(\mathbf{a}, \mathbf{b})$ to denote a multivariate random variable that is uniformly distributed on the hypercube defined by vector $\mathbf{a}$ with the lower boundaries and the vector $\mathbf{b}$ with the upper boundaries. The dimensions are typically clear from the context if not explicitly stated. Bold indicates a vector, e.g., $\boldsymbol{\theta} \in \mathbb{R}^D$.

Setup. We provide further details about the introductory example shown in Figure 1. The introductory example consists of five different versions: (1) Simple, (2) High-dimensional, (3) Distractor, (4) Multiple observations, and (5) Combined.

The simulator of the Simple (1), High-dimensional (2) and Multiple observations (4) versions is:

$$\mathbf{y} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I})) \quad (15)$$

At the Distractor (3) and the Combined (5) versions, we add $D_{\text{dist}} = 10$ distractor dimensions to the output, so the simulator becomes:

$$(\mathbf{y}^{(1)}, \mathbf{y}^{(2)}), \quad \mathbf{y}^{(1)} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \mathbf{I})), \quad \mathbf{y}^{(2)} \sim \mathcal{N}(\boldsymbol{\mu}, 100\mathbf{I}), \quad (16)$$

where $\boldsymbol{\mu} \sim \mathcal{U}(-10, 10)$. In all cases the prior is given by $p(\boldsymbol{\theta}) = \mathcal{U}(-15, 15)$.

C.1 Simple Case

The simulator is the one in (15) with $D = 5$ and the observation is $\mathbf{y}^o = (5, \dots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^5$. The ground truth posterior for $\mathbf{y}^o = (5, \dots, 5)$ is given by:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \propto p(\boldsymbol{\theta})p(\mathbf{y}^o|\boldsymbol{\theta}) \quad (17)$$

$$\propto p(\boldsymbol{\theta})(\mathcal{N}(\mathbf{y}^o; \boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(\mathbf{y}^o; -\boldsymbol{\theta}, \mathbf{I})) \quad (18)$$

$$\propto p(\boldsymbol{\theta})(\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}^o, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}^o, \mathbf{I})) \quad (19)$$

$$\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^5, \\ 0, & \text{otherwise.} \end{cases} \quad (20)$$

From (18) to (19), we apply the property: $\mathcal{N}(\mathbf{y}; -\boldsymbol{\theta}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}, \Sigma)$.

C.2 High-dimensional Case

The simulator is the one in (15) with $D = 20$ and the observation is $\mathbf{y}^o = (5, \dots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^{20}$. The derivation of (20) holds irrespectively of the dimensionality, so the ground truth posterior for $\mathbf{y}^o = (5, \dots, 5)$ is:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^{20}, \\ 0, & \text{otherwise.} \end{cases} \quad (21)$$

C.3 Distractor Case

The simulator is the one in (16) with $D = 5$ and $D_{\text{dist}} = 10$, leading to an output dimensionality of $D_y + D_{\text{dist}} = 15$. The observation is $\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})$, where $\mathbf{y}^{o,(1)} = (5, \dots, 5) + \epsilon$ and $\mathbf{y}^{o,(2)} = (0, \dots, 0) + \epsilon$. Below, we show

that the ground truth posterior is equivalent to the simple case (20), as the distractor dimensions do not affect the posterior:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \propto p(\boldsymbol{\theta})p(\mathbf{y}^o|\boldsymbol{\theta}) \quad (22)$$

$$\propto p(\boldsymbol{\theta})p((\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})|\boldsymbol{\theta}) \quad (23)$$

$$\propto p(\boldsymbol{\theta})p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta})p(\mathbf{y}^{o,(2)}|\boldsymbol{\theta}) \quad (24)$$

$$\propto p(\boldsymbol{\theta})p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta})p(\mathbf{y}^{o,(2)}) \quad (25)$$

$$\propto p(\boldsymbol{\theta})p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta}) \quad (26)$$

$$\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^5, \\ 0, & \text{otherwise.} \end{cases} \quad (27)$$

From (23) to (24), we use that $\mathbf{y}^{o,(1)}$ is independent of $\mathbf{y}^{o,(2)}$ and from (24) to (25), that $\mathbf{y}^{o,(2)}$ is independent of $\boldsymbol{\theta}$.

C.4 Multiple Observations Case

The simulator is the one in (15) with $D = 5$ and the set of $N = 4$ iid observations is $\{\mathbf{y}_n^o\}_n^N$ where $\mathbf{y}_n^o = (5, \dots, 5) + \epsilon$ for $n = 1, 2$ and $\mathbf{y}_n^o = (-5, \dots, -5) + \epsilon$ for $n = 3, 4$. We will derive the ground truth posterior for the general case of N observations, where $\mathbf{y}_n^o = (5, \dots, 5) + \epsilon$ for $n \in \{1, \dots, N/2\}$ and $\mathbf{y}_n^o = (-5, \dots, -5) + \epsilon$ for $n \in \{N/2+1, \dots, N\}$.

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto p(\boldsymbol{\theta}) \prod_n^N p(\mathbf{y}_n^o|\boldsymbol{\theta}) \quad (28)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^{N/2} p(\mathbf{y}_n^o|\boldsymbol{\theta}) \prod_{n=N/2+1}^N p(\mathbf{y}_n^o|\boldsymbol{\theta}) \quad (29)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^{N/2} (\mathcal{N}(\mathbf{5}; \boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\mathbf{5}; \boldsymbol{\theta}, \mathbf{I})) \prod_{n=N/2+1}^N (\mathcal{N}(-\mathbf{5}; \boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(\mathbf{5}; \boldsymbol{\theta}, \mathbf{I})) \quad (30)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N (\mathcal{N}(\mathbf{5}; \boldsymbol{\theta}, \mathbf{I}) + \mathcal{N}(-\mathbf{5}; \boldsymbol{\theta}, \mathbf{I})) \quad (31)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N (\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})) \quad (32)$$

$$\propto p(\boldsymbol{\theta}) (\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}))^N + (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}))^N \quad (33)$$

$$\propto p(\boldsymbol{\theta}) (\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I})) \quad (34)$$

$$\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^5, \\ 0, & \text{otherwise.} \end{cases} \quad (35)$$

From (29) to (30), we simply plug in the corresponding observations. From (31) to (32), we apply the property: $\mathcal{N}(\mathbf{y}; -\boldsymbol{\theta}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}, \Sigma)$. From (32) to (33), we use that the terms that include the product $\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})$ have a much smaller magnitude than the two terms $\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})^N$ and $\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})^N$. Therefore, we keep only the latter in (33). From (33) to (34), we use the property: $\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\alpha}, \Sigma)^N = \mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\alpha}, \frac{1}{N}\Sigma)$.

C.5 Combined Case

The simulator is the one in (16) with $D = 20$ and $D_{\text{dist}} = 10$, leading to an output dimensionality of $D_y + D_{\text{dist}} = 30$. The set of $N = 4$ iid observations is $\{\mathbf{y}_n^o\}_n^N$ where $\mathbf{y}_n^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})$, with $\mathbf{y}^{o,(1)}$ equal $(5, \dots, 5) + \epsilon$ for $n \in \{1, \dots, N/2\}$ and $(-5, \dots, -5) + \epsilon$ for $n \in \{N/2+1, \dots, N\}$ and $\mathbf{y}^{o,(2)}$ equal $(0, \dots, 0) + \epsilon$ for all n .

Using the properties of the previous cases we obtain the ground truth posterior, which is the same as in the

multiple observations case (35) for $D = 20$:

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}_n^o|\boldsymbol{\theta}) \quad (36)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p((\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)})|\boldsymbol{\theta}) \quad (37)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta})p(\mathbf{y}^{o,(2)}|\boldsymbol{\theta}) \quad (38)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta})p(\mathbf{y}^{o,(2)}) \quad (39)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}^{o,(1)}|\boldsymbol{\theta}) \quad (40)$$

From (37) to (38), we use that the observations are independent and from (38) to (39) that $\mathbf{y}^{o,(2)}$ is independent of $\boldsymbol{\theta}$. Using the derivation of the multiple observations case (35) gives

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N}\mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N}\mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^{20}, \\ 0, & \text{otherwise.} \end{cases} \quad (41)$$

C.6 Results

Setup. For NAIVE-ROMC and R2OMC, a maximum of $S = 5000$ seeds was used, corresponding to approximately 5000 independent simulator calls, which is (approximately) a simulation budget of 5000. NLE and NPE were trained with a simulation budget of 10000 samples. For NPE, we used $R = 2$ rounds with 5000 samples each, while for NLE, we used a single round of 10000 samples. The default settings from the SBI library Lueckmann et al. (2021) were utilized for both, with a training batch size of 1000 and a maximum of 300 training epochs.

Conclusions. The results of the introductory example are presented in Figure 1 in the main text. In the simple scenario, all methods provide a reasonably accurate approximation of the ground truth posterior. In the high-dimensional case, both NPE and NLE encounter difficulties, indicating that high-dimensional problems are challenging for these methods even when the simulator is relatively simple. In contrast, NAIVE-ROMC and R2OMC yield a good approximation of the ground truth posterior, demonstrating their robustness in high-dimensional settings. Note that in the simple scenario, NAIVE-ROMC and R2OMC are equivalent, since there are no distractors and only one observation. In the distractors scenario, NPE struggles, NLE is more robust than NPE, but still faces challenges in providing an accurate approximation, while NAIVE-ROMC struggles. R2OMC provides a good approximation of the ground truth posterior. In the multiple observations scenario, only NLE and R2OMC succeed in generating a reliable approximation of the ground truth posterior. Finally, in the combined scenario, only R2OMC delivers a satisfactory approximation of the ground truth posterior.

D Additional information on the experiments

We here provide some additional information on the experiments presented in the main text.

D.1 MoG Benchmark - High Dimensional Simulators

D.1.1 Base simulator - without distractors

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)$
Simulator	$\mathbf{y} \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{I})$, with $\sigma^2 = 1.5^2$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^D$
Observation	$\mathbf{y}^o = (5, \dots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^D$
Posterior	$p(\boldsymbol{\theta} \mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

Proof of the posterior:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \propto p(\boldsymbol{\theta})p(\mathbf{y}^o|\boldsymbol{\theta}) \quad (42)$$

$$\propto p(\boldsymbol{\theta})\mathcal{N}(\mathbf{y}^o; \boldsymbol{\theta}, \sigma^2 \mathbf{I}) \quad (43)$$

$$\propto p(\boldsymbol{\theta})\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}^o, \sigma^2 \mathbf{I}) \quad (44)$$

$$\propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases} \quad (45)$$

From (43) to (44), we apply the property: $\mathcal{N}(\mathbf{y}; -\boldsymbol{\theta}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; -\mathbf{y}, \Sigma)$.

D.1.2 Base simulator - with distractors

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)$
Simulator	$\mathbf{y} = (\mathbf{y}^{(1)}, \mathbf{y}^{(2)}),$ $\mathbf{y}^{(1)} \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{I}), \sigma^2 = 1.5^2,$ $\mathbf{y}^{(2)} \sim \mathcal{N}(\boldsymbol{\mu}, 100 \mathbf{I}), \boldsymbol{\mu} \sim \mathcal{U}(-10, 10)$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^{D+100}, \mathbf{y}^{(1)} \in \mathbb{R}^D, \mathbf{y}^{(2)} \in \mathbb{R}^{100}$
Observation	$\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)}),$ where $\mathbf{y}^{o,(1)} = (5, \dots, 5) + \epsilon$ and $\mathbf{y}^{o,(2)} = (0, \dots, 0) + \epsilon$
Posterior	$p(\boldsymbol{\theta} \mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

The ground truth posterior is the same as in (45), as the distractors do not affect the posterior, see (27).

D.1.3 MoG simulator - without distractors

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)$
Simulator	$\mathbf{y} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2 \mathbf{I})), \sigma_1^2 = 0.1, \sigma_2^2 = 1$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^D$
Observation	$\mathbf{y}^o = (5, \dots, 5) + \epsilon$ where $\mathbf{y}^o \in \mathbb{R}^D$
Posterior	$p(\boldsymbol{\theta} \mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \sigma_2^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

The MoG simulator is the same as in the introductory example, so the proof is equivalent to Section C.

D.1.4 MoG simulator - with distractors

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)$
Simulator	$\mathbf{y} = (\mathbf{y}^{(1)}, \mathbf{y}^{(2)}),$ $\mathbf{y}^{(1)} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2 \mathbf{I})), \sigma_1^2 = 0.1, \sigma_2^2 = 1,$ $\mathbf{y}^{(2)} \sim \mathcal{N}(\boldsymbol{\mu}, 100 \mathbf{I}), \boldsymbol{\mu} \sim \mathcal{U}(-10, 10)$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^{D+100}, \mathbf{y}^{(1)} \in \mathbb{R}^D, \mathbf{y}^{(2)} \in \mathbb{R}^{100}$
Observation	$\mathbf{y}^o = (\mathbf{y}^{o,(1)}, \mathbf{y}^{o,(2)}),$ where $\mathbf{y}^{o,(1)} = (5, \dots, 5) + \epsilon$ and $\mathbf{y}^{o,(2)} = (0, \dots, 0) + \epsilon$
Posterior	$p(\boldsymbol{\theta} \mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \sigma_2^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

The MoG simulator is the same as in the introductory example, so the proof is equivalent to Section C.

D.2 MoG Benchmark - Multiple Observations

D.2.1 Base simulator

Prior	$\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)$
Simulator	$\mathbf{y} \sim \mathcal{N}(\mathbf{y}; \boldsymbol{\theta}, \sigma^2 \mathbf{I}), \sigma^2 = 1$
Dimensionality	$\boldsymbol{\theta} \in \mathbb{R}^D, \mathbf{y} \in \mathbb{R}^D$
Observations	$\{\mathbf{y}_n^o\}_n^N$ where $\mathbf{y}_n^o = (5, \dots, 5) + \epsilon$ for $n \in \{1, \dots, 4\}$
Posterior	$p(\boldsymbol{\theta} \mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N} \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

Proof of the posterior:

$$p(\boldsymbol{\theta} | \{\mathbf{y}_n^o\}) \propto p(\boldsymbol{\theta}) \prod_n^N p(\mathbf{y}_n^o | \boldsymbol{\theta}) \quad (46)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N \mathcal{N}(\mathbf{y}_n^o; \boldsymbol{\theta}, \sigma^2 \mathbf{I}) \quad (47)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \sigma^2 \mathbf{I}) \quad (48)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \sigma^2 \mathbf{I}) \quad (49)$$

$$\propto \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N} \sigma^2 \mathbf{I}) \quad (50)$$

$$\propto \begin{cases} p(\boldsymbol{\theta}) \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N} \sigma^2 \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases} \quad (51)$$

In Figure 8, we present additional results for the base simulator with multiple observations.

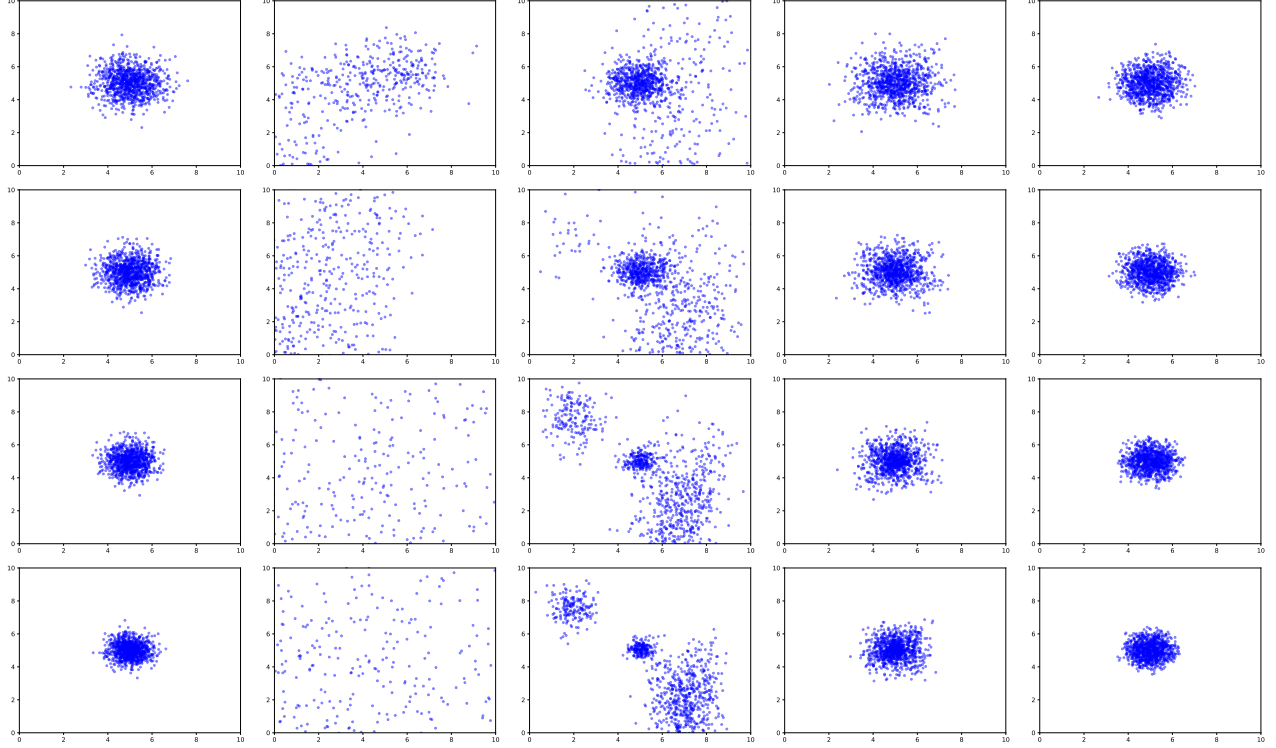


Figure 8: Multi observations Base: Left to right: ground truth, NPE, NLE, Naive-ROMC, R2OMC. Top to bottom: $N = 3$, $N = 5$, $N = 10$, $N = 10$

D.2.2 MoG simulator

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)$

Simulator $\mathbf{y} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}, \sigma_1^2 \mathbf{I}) + \mathcal{N}(-\boldsymbol{\theta}, \sigma_2^2 \mathbf{I}))$, with $\sigma_1 = \sigma_2 = 1$

Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^D$, $\mathbf{y} \in \mathbb{R}^D$

Observations $\{\mathbf{y}_n^o\}_n^N$ where:

$$\mathbf{y}_n^o = (5, \dots, 5) + \epsilon \text{ for } n \in \{1, \dots, \frac{N}{2}\},$$

$$\mathbf{y}_n^o = (-5, \dots, -5) + \epsilon \text{ for } n \in \{\frac{N}{2} + 1, \dots, N\}$$

Posterior $p(\boldsymbol{\theta}|\mathbf{y}^o) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N} \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N} \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases}$

Proof of the posterior:

$$p(\boldsymbol{\theta}|\{\mathbf{y}_n^o\}) \propto \begin{cases} \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \frac{1}{N} \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N} \mathbf{I}), & \text{if } \boldsymbol{\theta} \in [-15, 15]^D, \\ 0, & \text{otherwise.} \end{cases} \quad (52)$$

The proof is similar to the one in Eq. (35).

In Figure 9, we present the results for the MoG simulator with multiple observations.

D.3 SBI - Mixtures of Gaussians (T.7)

We here provide additional information on example T.7 of the SBI benchmark [Lueckmann et al. \(2021\)](#). It tests inference of the mean of a mixture of two 2-D Gaussians, one with much broader covariance than the other

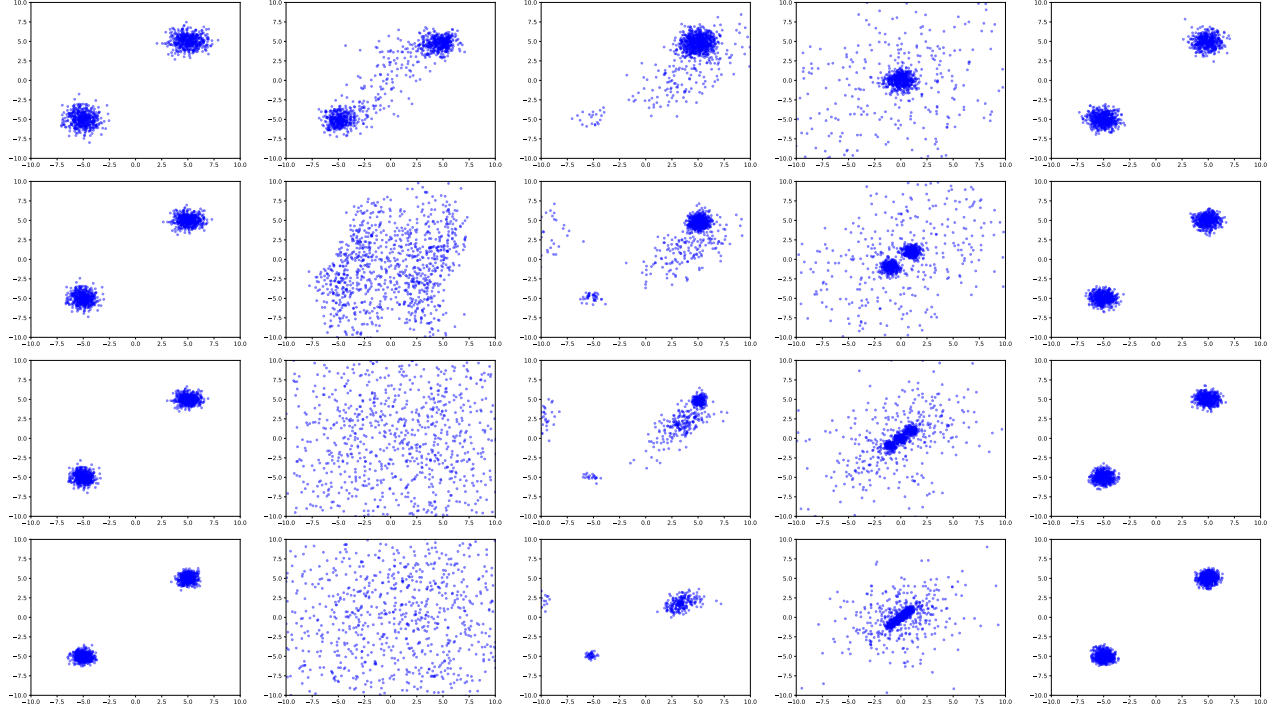


Figure 9: Multi observations MoG: Left to right: ground truth, NPE, NLE, Naive-ROMC, R2OMC. Top to bottom: $N = 3$, $N = 5$, $N = 10$, $N = 10$

Prior $\theta \sim \mathcal{U}(-10, 10)$
Simulator $\mathbf{y} \sim 0.5(\mathcal{N}(\theta, \mathbf{I}) + \mathcal{N}(\theta, \sigma^2 \mathbf{I}))$, with $\sigma^2 = 0.01$
Dimensionality $\theta \in \mathbb{R}^2, \mathbf{y} \in \mathbb{R}^2$

Results. We evaluate R2OMC using a simulation budget of $S = 1000$ and $S = 10000$ seeds. The C2ST score is computed by comparing 1000 samples from the true posterior (we randomly select 1000 from the 10000 that are provided by the SBI benchmark) with 1000 samples generated from the R2OMC approximate posterior. The results are illustrated in Figure 10.

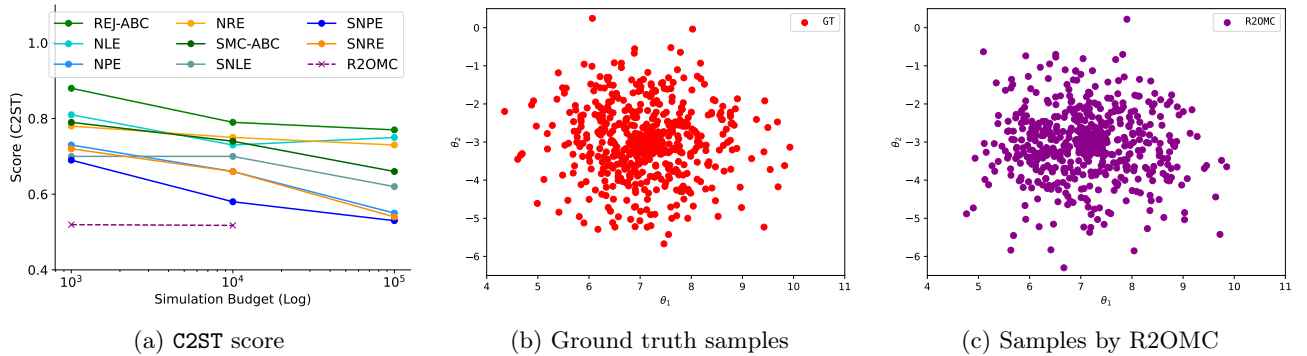


Figure 10: T.7: Gaussian Mixture. From left to right: C2ST score, ground truth samples, and samples by R2OMC.

D.4 SBI - Two Moons (T.8)

We here provide additional information on example T.8 of the SBI benchmark [Lueckmann et al. \(2021\)](#). It tests inference when the posterior exhibits both global (bimodality) and local (crescent shape) structure to illustrate

how algorithms deal with multimodality:

Prior $\theta \sim \mathcal{U}(-1, 1)$

Simulator $\mathbf{y} \sim \begin{bmatrix} r \cos(\alpha) + 0.25 \\ r \sin(\alpha) \end{bmatrix} + \begin{bmatrix} |\theta_1 + \theta_2|/\sqrt{2} \\ (-\theta_1 + \theta_2)/\sqrt{2} \end{bmatrix}$, where $r \sim \mathcal{N}(0.1, 0.01^2)$ and $\alpha \sim \mathcal{U}(-\pi/2, \pi/2)$

Dimensionality $\theta \in \mathbb{R}^2, \mathbf{y} \in \mathbb{R}^2$

Results. We evaluate R2OMC using a simulation budget of $S = 1000$ and $S = 10000$ seeds. The C2ST score is computed by comparing 1000 samples from the true posterior (we randomly select 1000 from the 10000 that are provided by the SBI benchmark) with 1000 samples generated from the R2OMC approximate posterior. The results are illustrated in Figure 10. We observe that R2OMC is able to capture the bimodal structure of the posterior, perfectly replicating the ground truth samples.

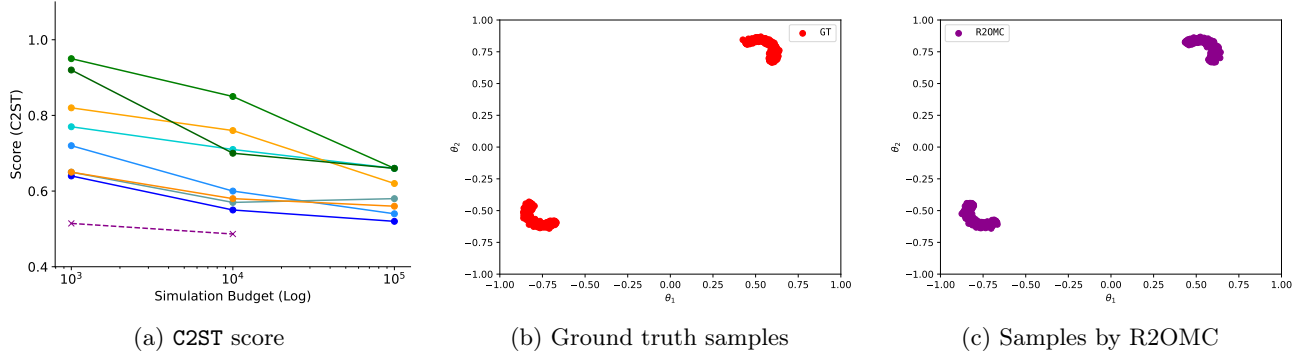


Figure 11: T.8: Two Moons. From left to right: C2ST score, ground truth samples, and samples by R2OMC.

E Additional results

We here present some additional experiments that were not included in the main text.

E.1 MoG - Shifted Gaussian

This additional experiment uses a simulator that samples from a mixture of two Gaussians, whose means are shifted by a constant value.

Prior $\boldsymbol{\theta} \sim \mathcal{U}(-15, 15)^4$
Simulator $\mathbf{y} \sim \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta} + \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}, \mathbf{I}))$
Dimensionality $\boldsymbol{\theta} \in \mathbb{R}^4, \mathbf{y} \in \mathbb{R}^4$

We will use two separate cases: one with $N = 4$ similar observations and another with $N = 4$ different observations. In both cases, the posterior is:

$$p(\boldsymbol{\theta} | \{\mathbf{y}_n^o\}) \propto p(\boldsymbol{\theta}) p(\{\mathbf{y}_n^o\} | \boldsymbol{\theta}) \quad (53)$$

$$\propto p(\boldsymbol{\theta}) \prod_{n=1}^N p(\mathbf{y}_n^o | \boldsymbol{\theta}) \quad (54)$$

$$\propto \mathcal{U}(-15, 15) \prod_{n=1}^N \frac{1}{2} (\mathcal{N}(\mathbf{y}_n^o; \boldsymbol{\theta} + \mathbf{5}, \mathbf{I}) + \mathcal{N}(\mathbf{y}_n^o; \boldsymbol{\theta}, \mathbf{I})) \quad (55)$$

$$\propto \mathcal{U}(-15, 15) \prod_{n=1}^N \frac{1}{2} (\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})) \quad (56)$$

$$\propto \prod_{n=1}^N (\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})) \quad (57)$$

In (54), the factorization is due to the N independently and identically distributed (iid) data points. In (56), we apply the property: $\mathcal{N}(\mathbf{y}; \boldsymbol{\theta} + \boldsymbol{\alpha}, \Sigma) = \mathcal{N}(\boldsymbol{\theta}; \mathbf{y} - \boldsymbol{\alpha}, \Sigma)$. In (57), we use that the data that we will use are sufficiently far from the boundaries so that we may ignore the boundary effects in the truncated distribution.

Case 1: In Case 1, we use $N = 4$ similar observations: $\mathbf{y}_n^o \approx (0, \dots, 0)$ for $n \in \{1, \dots, N\}$. The ground-truth posterior is approximately:

$$p(\boldsymbol{\theta} | \mathbf{y}^o) \approx \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N} \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N} \mathbf{I}) \quad (58)$$

Proof.

$$p(\boldsymbol{\theta} | \mathbf{y}_n^o) \propto \prod_n^N (\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})) \quad (59)$$

$$\propto (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^N \quad (60)$$

$$\propto \sum_{n=0}^N \binom{N}{n} (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}))^n (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^{N-n} \quad (61)$$

$$\propto (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}))^N + (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^N \quad (62)$$

$$\propto \mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \frac{1}{N} \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N} \mathbf{I}) \quad (63)$$

In (62), for $n \in \{1, \dots, N-1\}$ the terms involve products between $\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})$ and $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})$ so they have a much smaller magnitude compared to $(\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}))^N$ ($n = N$) and $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^N$ ($n = 0$). This allows us to approximate the expression by retaining only these two dominant terms. In (63), we use the property that raising a Gaussian distribution to the power N results in a Gaussian with the same mean and a covariance scaled by $\frac{1}{N}$.

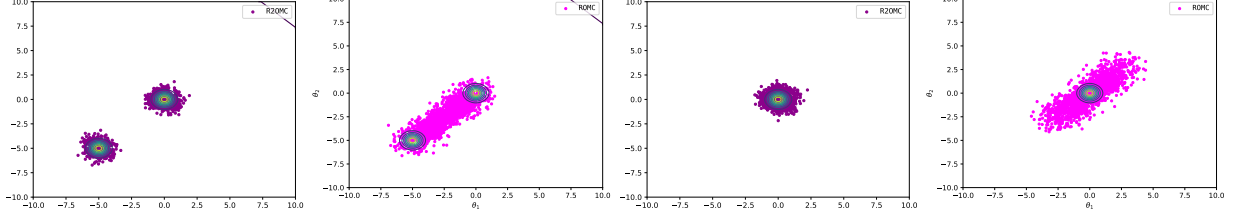


Figure 12: From left to right: R2OMC on Case 1, Naive-ROMC on Case 1, R2OMC on Case 2, and Naive-ROMC on Case 2.

Case 2: In Case 2, we use an even number of N observations that are split in two categories, half around $(0, \dots, 0)$, i.e., $\mathbf{y}_n^o \approx (0, \dots, 0)$ for $n \in \{1, \dots, N/2\}$ and the other half around $(5, \dots, 5)$, i.e., $\mathbf{y}_n^o \approx (5, \dots, 5)$ for $n \in \{N/2 + 1, \dots, N\}$. The posterior, for values of $\boldsymbol{\theta}$ sufficiently far from the boundary, is:

$$p(\boldsymbol{\theta}|\mathbf{y}^o) \approx \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N}\mathbf{I}) \quad (64)$$

Proof.

$$p(\boldsymbol{\theta}|\{\mathbf{y}_k^o\}) \propto \prod_n^N (\mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o - \mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{y}_n^o, \mathbf{I})) \quad (65)$$

$$\propto \prod_{n=1}^{N/2} (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})) \prod_{n=N/2+1}^N (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})) \quad (66)$$

$$\propto \prod_{n=1}^{N/2} (\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})) \prod_{n=1}^{N/2} (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}) + \mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})) \quad (67)$$

$$\propto \left((\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I}))^{N/2} + (\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^{N/2} \right) \left((\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I}))^{N/2} + (\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I}))^{N/2} \right) \quad (68)$$

$$\propto \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^N \quad (69)$$

$$\propto \mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \frac{1}{N}\mathbf{I}) \quad (70)$$

In (68), we keep only the dominant terms, as in Case 1. In (69), we use that terms that involve combinations of $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^{N/2}$, $\mathcal{N}(\boldsymbol{\theta}; -\mathbf{5}, \mathbf{I})^{N/2}$ and $\mathcal{N}(\boldsymbol{\theta}; \mathbf{5}, \mathbf{I})^{N/2}$ have significantly smaller magnitudes compared to the dominant ones; $\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^{N/2}\mathcal{N}(\boldsymbol{\theta}; \mathbf{0}, \mathbf{I})^{N/2}$. In (70), we use the property that raising a Gaussian distribution to the power N results in a Gaussian with the same mean and a covariance scaled by $\frac{1}{N}$.

Results. Figure 12 demonstrates that R2OMC aligns closely with the ground truth posterior, correctly capturing the two modes in Case 1 and the single mode in Case 2. In contrast, both ROMC struggles with multiple observations, resulting in their samples being distributed along the 45° line.